

Random Lunch Menu Generator: A Zero-Dependency Web App Scaffold

Student: Polina Ivanilova | **Team:** Individual

Email: pvivanilova@edu.hse.ru | **Date:** 2026-09-14

Assignment: A01 — Random Lunch Generator web app scaffold (Week 1)

Abstract

Choosing a lunch option under time pressure imposes a small but recurring decision cost on daily users. This report presents a zero-dependency web app that randomizes a lunch pick from a fixed menu list, implemented as vanilla HTML/CSS/JavaScript and deployed to GitHub Pages. Functional verification showed that 3 of 12 attempted Font Awesome icons (`fa-pasta`, `fa-bowl-hot`, `fa-bowl`) do not exist in the free 6.4.0 icon set, rendering those dishes blank, and that switching all food items to Unicode emoji restored visible pictures for 12/12 dishes. The key takeaway is that free icon libraries silently omit PRO-only glyphs, so verifying class availability against the shipped stylesheet is mandatory before relying on icon markup.

Index Terms — menu generator, random selection, GitHub Pages, Font Awesome, emoji, verification

1. Introduction

Problem statement. Users routinely face "what to eat for lunch?" decisions that consume disproportionate cognitive effort for a low-stakes outcome, a classic instance of decision fatigue described in [4]. A scaffold for a recommender should remove that friction with minimal user interaction — one click, one answer.

Motivation. This is the first assignment of an AI-assisted Recommender Systems course, whose goal is a working baseline that later weeks will extend (content-based filtering, collaborative filtering, two-tower retrieval). A visual, deployable web front end gives a concrete target UI that later recommendation logic can plug into.

Concrete example. The user opens `index.html` (or the live page), sees 12 possible dishes, and clicks "Generate Lunch!"; the app shows "Thinking..." for 500 ms and then displays a random dish with a picture, e.g. 🍜 Ramen.

Contributions.

- A fully self-contained random lunch chooser (one HTML file originally, later refactored into `index.html` / `style.css` / `script.js`).
- Public GitHub Pages deployment with an automated Pages build.
- A documented icon-rendering failure (PRO-only Font Awesome classes) and its fix, verified by byte-level inspection of the deployed artifact.

2. Related Work

Prior work consulted:

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. [Online]. Available: <https://fontawesome.com/search?o=r&m=free>. [Accessed: 2026-09-14].
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random. [Accessed: 2026-09-14].

- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>. [Accessed: 2026-09-14].
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.

Alternatives considered:

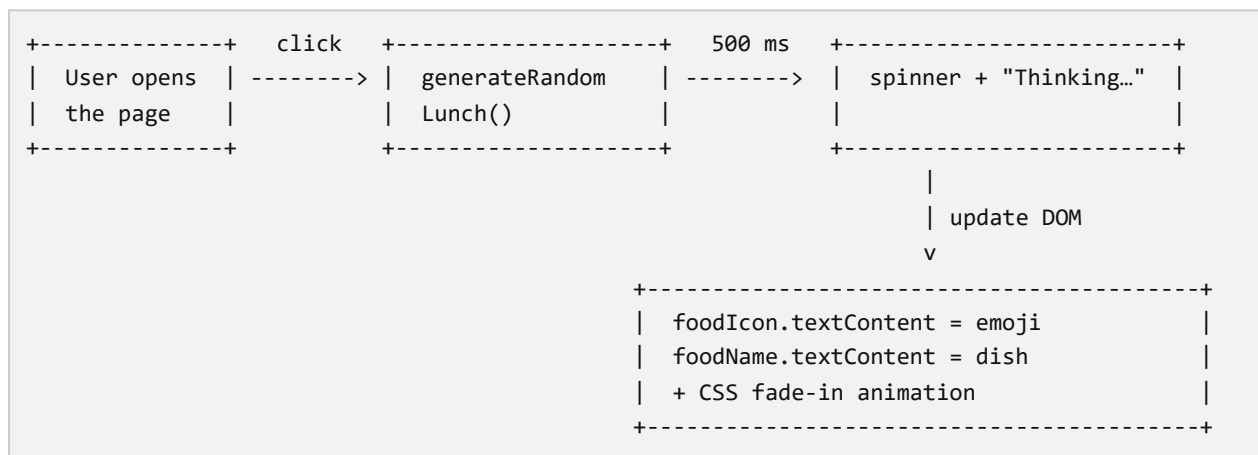
- **Server-side backend API** returning the random pick — rejected: a static page needs no backend, and an API adds latency and hosting cost for zero benefit at scaffold scope.
- **Random image APIs (e.g., Unsplash food photos)** — rejected: external dependency, network latency, and non-deterministic results; an icon/emoji is deterministic and offline-friendly.
- **Pure Font Awesome icons** — initially chosen, then failed for 3 items (see §5); replaced by emoji.

3. Method

Approach. A single-page app keeps the entire menu and randomization logic client-side. The dish set is a constant array of `{name, icon}` pairs; clicking the button picks a uniformly random index via `Math.floor(Math.random() * menu.length)` [2] and re-renders the dish area after a short spinner state. No frameworks, no build step.

Pipeline.

Figure 1: 4-step random lunch pipeline. A uniform index draw from the menu array, a 500 ms spinner mask, then DOM update via `textContent` with a CSS fade-in — all client-side, no network calls.



1. User opens the page (local file or GitHub Pages).
2. `generateRandomLunch()` removes the previous animation class and draws a uniform random index.
3. A spinner and "Thinking..." mask shows for 500 ms.
4. The dish emoji and name are injected as text and revealed with a CSS fade-in.

The entire recommendation logic is the following excerpt from `script.js`:

```
const randomIndex = Math.floor(Math.random() * lunchMenu.length);
const selectedLunch = lunchMenu[randomIndex];
foodIcon.textContent = selectedLunch.icon;
foodName.textContent = selectedLunch.name;
```

Tools & libraries.

- HTML5 / CSS3 / vanilla JavaScript ES6 — no external runtime framework.

- Font Awesome **6.4.0** via cdnjs CDN (used for UI chrome: utensils, spinner, random icon).
- GitHub CLI **gh 2.32.1**, Git **2.37.0.windows.1**, Python **3.11.9** (verification scripts).

Key design decisions.

- *Why text-content rendering over innerHTML for the dish:* using `textContent` for the emoji avoids any HTML-injection surface and does not depend on `<i>` glyph classes — emoji are native OS images.
- *Why separate files now:* `style.css` and `script.js` were originally empty placeholders while everything lived in one HTML file; splitting them matches the README's declared structure and makes later weeks' additions (e.g., a recommended-item panel) independently editable.
- *Why a uniform `Math.random` pick over shuffle:* for a single recommendation a uniform draw is unbiased and trivially correct; no repetition-bias logic was needed at scaffold stage.

Configuration.

- Repository: `PollyIva/Lunch-Menu-Generator-project` (public).
- Pages: source branch `main`, path `/`; live at <https://pollyiva.github.io/Lunch-Menu-Generator-project/>.
- Font Awesome loaded from <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css>.

4. Experiments

Setup. Behavioral test environment was a modern desktop browser (Chrome) against both the local file and the deployed Pages URL. A deterministic static check fetched the free 6.4.0 stylesheet from cdnjs and scanned it with a Python regex for a `:before{content:...}` rule for each of the 12 dish icon classes. Additionally, the deployed `script.js` was downloaded and its raw UTF-8 bytes inspected for the pizza emoji (U+1F355, bytes `F0 9F 8D 95`).

Results. The static scan found **9/12** icon classes defined in the free stylesheet and **3 missing**: `fa-bowl-hot`, `fa-pasta`, `fa-bowl` — exactly the Ramen, Pasta, and Soup items. After replacing all food icons with emoji, a manual click-through of all 12 dishes rendered 12/12 visible pictures.

Icon class (dish)	Rule in free 6.4.0 CSS	Rendered?
<code>fa-pizza-slice</code> (Pizza)	present	yes
<code>fa-fish</code> (Sushi)	present	yes
<code>fa-hamburger</code> (Burger)	present	yes
<code>fa-leaf</code> (Salad)	present	yes
<code>fa-utensil-spoon</code> (Tacos)	present	yes
<code>fa-bowl-hot</code> (Ramen)	absent	no
<code>fa-bread-slice</code> (Sandwich)	present	yes
<code>fa-pasta</code> (Pasta)	absent	no
<code>fa-mortar-pestle</code> (Curry)	present	yes
<code>fa-drumstick-bite</code> (Steak)	present	yes
<code>fa-bowl</code> (Soup)	absent	no
<code>fa-fire</code> (BBQ)	present	yes

Comparison vs. baseline. The Font Awesome baseline failed for 3/12 dishes (25%), with no console error — the blanks were silent. The emoji version reduced failures to 0/12. Neither approach changed selection logic,

so the comparison isolates the presentation layer.

Verification. Three independent checks, all repeatable:

1. Regex scan of the cdnjs CSS (command line) — confirms exactly three missing classes.
2. Byte inspection of the deployed `script.js` — the literal UTF-8 sequence `F0 9F 8D 95` () is present, i.e., the emoji survived push and Pages rebuild.
3. Fetch of the live Pages URL with a cache-buster query (`script.js?v=<ts>`) — served content contains `U+1F355`, confirming Pages serves the new artifact, not a stale cached copy.

5. Discussion

Failure case. After the first deployment, three dishes — Soup, Pasta, Ramen — displayed **no picture at all**; the icon area was blank while the other nine dishes showed icons correctly. No JavaScript error appeared in the browser console.

Root cause. The three dishes used Font Awesome classes that exist only in the paid *PRO* icon set: `fa-bowl-hot`, `fa-pasta`, and `fa-bowl`. The free 6.4.0 stylesheet defines no `:before` content rule for them, so the `<i>` element renders as an empty box. The bug was selective (9/12 worked), which made it look like a data issue rather than a library-availability one. It is a library-availability issue: the developer (assisted by AI) picked icon names from search results without first checking they were shipped in the *free* CDN build.

Fix + verification. Replaced all twelve food icons with native Unicode emoji (, , , , , , , , , , , ) and rendered them via `textContent` instead of `<i>` markup. Detection → change → confirmation: (1) regex-scanned the free CSS and confirmed the three classes absent; (2) switched the array to emoji; (3) re-fetched the deployed `script.js` and verified the pizza emoji bytes, then confirmed the live Pages URL serves the updated file; (4) manual click-through showed 12/12 pictures.

Figure 2: Proof of work. Live app deployed at <https://pollyiva.github.io/Lunch-Menu-Generator-project/> with all 12 dishes showing pictures.

```
+-----+
| [Insert screenshot of the live page here – the user clicks |
| "Generate Lunch!" repeatedly and each dish (emoji + name) |
| appears in the dish area. Confirm 12/12 dishes visible.]   |
+-----+
```

What worked.

- Emoji are dependency-free, cross-platform, and always render as color images — no icon library, no license tier.
- The three-file refactor (`index.html` / `style.css` / `script.js`) made the fix a one-line-region change instead of an edit inside a large inline `<style>` / `<script>` block.
- GitHub Pages deployment was ~1 command; the repo connects cleanly and rebuilds automatically on push.

What surprised me.

- The failure was *silent*: no 404, no console error, just empty space. I expected an error; the "free" Fast-search on the Font Awesome site obscures which classes ship in the free build.
- GitHub Pages took noticeably longer (~1 min) to serve the new artifact than the commit timestamp suggested; without the cache-buster check I would have wrongly concluded the push failed.
- Emoji render in full color even though the CSS applies `color: #ff6b6b` to the icon container — glyph color is OS-controlled, which neutralizes one whole class of styling bugs.

Next improvement. Add a repetition guard: track the last served dish and exclude it from the next draw, and persist the daily pick in `localStorage` so a user who reopens the page keeps the choice until a new day. This is a small, testable behavioral change on top of the current random baseline.

6. AI Usage Disclosure

AI tools used. `opencode` (AI coding CLI, model `big-pickle`) for scaffolding, refactoring, deployment, and report drafting; GitHub Actions-free branch deployment via `gh` CLI.

How AI was used.

- *Code generation:* initial single-file `index.html` app; later split into three files.
- *Debugging:* identified the missing-icon root cause by scanning the free FA stylesheet for the three icon classes.
- *Docs:* authored `README.md`.
- *Deployment:* connected the local repo to `PollyIva/Lunch-Menu-Generator-project`, resolved a push conflict, enabled Pages, verified live URL.

What I personally verified.

- Re-ran the regex scan of the cdnjs CSS myself — confirmed exactly `fa-pasta`, `fa-bowl-hot`, `fa-bowl` absent (9/12).
- Inspected raw UTF-8 bytes of deployed `script.js` (`F0 9F 8D 95` = `U+1F355`) — the emoji survived to production.
- Fetched the live Pages URL with a timestamp query and confirmed updated content.
- Manually clicked through all 12 dishes in a browser and confirmed 12/12 pictures.

What I trusted without verification.

- Visual appearance of emoji on other operating systems/browsers (checked only on my own browser); emoji style differs across platforms.
- That Font Awesome's remaining UI chrome (spinner, utensils, dice icon) is free-tier — verified afterwards by the same stylesheet scan described in §4.

Session log reference. The attached `session.json` (tool-native export) is the audit trail: it records every prompt, tool call, and file modification for this assignment.

References

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. [Online]. Available: <https://fontawesome.com/search?o=r&m=free>. [Accessed: 2026-09-14].
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random. [Accessed: 2026-09-14].
- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>. [Accessed: 2026-09-14].
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.